

MongoDB with Mongoose — Reference Guide

Connections, schemas/models, and CRUD operations using Mongoose — definitions, syntax, and examples.

1. Database Operations

Create / Connect to Database

Mongoose does not have a separate command to create a database ahead of time. Calling `mongoose.connect()` with a connection string that includes a database name selects (and implicitly creates) that database the first time a document is saved to it.

Syntax

```
mongoose.connect(<connectionString>, <options>)
```

Example

```
const mongoose = require("mongoose");

mongoose.connect("mongodb://localhost:27017/schoolDB")
  .then(() => console.log("Connected to schoolDB"))
  .catch((err) => console.error(err));
// "schoolDB" is selected, but only created on disk once
// a document is written to one of its collections.
```

Delete Database

`mongoose.connection.dropDatabase()` permanently deletes the database the current connection points to, including all of its collections and documents. This action cannot be undone.

Syntax

```
mongoose.connection.dropDatabase()
```

Example

```
mongoose.connection.dropDatabase()
  .then(() => console.log("schoolDB dropped"))
  .catch((err) => console.error(err));
// schoolDB and all its data are permanently removed
```

2. Schema & Model Operations

Create Schema & Model

In Mongoose, a Schema defines the shape and validation rules for documents, and a Model — created from that schema — is what you actually use to read and write to a collection. Defining a model is the Mongoose

equivalent of creating a collection: Mongoose derives the collection name automatically (a pluralized, lowercased version of the model name) and creates it the first time a document is saved.

Syntax

```
const schema = new mongoose.Schema(<definition>);
const Model = mongoose.model(<name>, schema);
```

Example

```
const studentSchema = new mongoose.Schema({
  name: String,
  grade: Number,
  subjects: [String]
});

const Student = mongoose.model("Student", studentSchema);
// Creates (or reuses) the "students" collection
```

Delete Collection

`Model.collection.drop()` permanently removes the underlying collection for a model, along with all of its documents and indexes. This action cannot be undone.

Syntax

```
Model.collection.drop()
```

Example

```
Student.collection.drop()
  .then(() => console.log("Collection dropped"))
  .catch((err) => console.error(err));
// The "students" collection and all its documents are removed
```

3. CRUD Operations on a Model

Create — `Model.create()`

Creates and saves one or more new documents to the collection in a single call. Accepts either a single object or an array of objects.

Syntax

```
Model.create(<document>)
```

Example

```
Student.create({
  name: "Rahul",
  grade: 9,
  subjects: ["Math", "Science"]
})
```

```
})  
  .then((student) => console.log(student))  
  .catch((err) => console.error(err));
```

Create — new Model() + save()

Alternatively, construct a new document instance and call `save()` on it. This form is useful when you want to run validation or modify the document before it's persisted.

Syntax

```
const doc = new Model(<fields>);  
doc.save().then(<callback>).catch(<callback>);
```

Example

```
const student = new Student({ name: "Meera", grade: 7 });  
student.save()  
  .then((saved) => console.log(saved))  
  .catch((err) => console.error(err));
```

Read — Model.find()

Retrieves all documents that match a given query filter, returning a Mongoose Query (which resolves to an array of documents). Calling `find()` with no arguments returns every document in the collection.

Syntax

```
Model.find(<filter>, <projection>)
```

Example

```
Student.find({ grade: 8 })  
  .then((students) => console.log(students))  
  .catch((err) => console.error(err));  
// Returns all documents where grade equals 8
```

Read — Model.findOne()

Retrieves the first document that matches a given query filter, resolving to a single document (or null) instead of an array.

Syntax

```
Model.findOne(<filter>, <projection>)
```

Example

```
Student.findOne({ name: "Rahul" })  
  .then((student) => console.log(student))  
  .catch((err) => console.error(err));  
// Returns the first matching document, or null if none match
```

Update — Model.findByIdAndUpdate()

The most commonly used update method in Mongoose apps — finds a document by its `_id` and updates it in one step. Pass `{ new: true }` to get the updated document back instead of the original.

Syntax

```
Model.findByIdAndUpdate(<id>, <update>, <options>)
```

Example

```
Student.findByIdAndUpdate(
  "64f1a2b3c4d5e6f7a8b9c0d1",
  { $set: { grade: 10 } },
  { new: true }
)
  .then((updated) => console.log(updated))
  .catch((err) => console.error(err));
```

Update — Model.findOneAndUpdate()

Finds the first document matching a filter and updates it in one step — useful when you don't have the `_id` but have some other unique field to match on. Like `findByIdAndUpdate()`, pass `{ new: true }` to receive the updated document.

Syntax

```
Model.findOneAndUpdate(<filter>, <update>, <options>)
```

Example

```
Student.findOneAndUpdate(
  { name: "Rahul" },
  { $set: { grade: 10 } },
  { new: true }
)
  .then((updated) => console.log(updated))
  .catch((err) => console.error(err));
```

Delete — Model.findByIdAndDelete()

The most commonly used delete method in Mongoose apps — finds a document by its `_id` and removes it in one step, resolving to the deleted document (or null if not found).

Syntax

```
Model.findByIdAndDelete(<id>)
```

Example

```
Student.findByIdAndDelete("64f1a2b3c4d5e6f7a8b9c0d1")
  .then((deleted) => console.log(deleted))
```

```
.catch((err) => console.error(err));
```

Delete — Model.findOneAndDelete()

Finds the first document matching a filter and removes it in one step — useful when you don't have the `_id` but have some other field to match on. Resolves to the deleted document, or null if nothing matched.

Syntax

```
Model.findOneAndDelete(<filter>)
```

Example

```
Student.findOneAndDelete({ name: "Kabir" })  
  .then((deleted) => console.log(deleted))  
  .catch((err) => console.error(err));
```

4. More About find()

find() with a Query Filter

A query filter narrows results to documents matching specific field values or query operators (e.g. `$gt`, `$lt`, `$in`). An empty filter (`{}`) matches every document.

Syntax

```
Model.find({ <field>: <value> })
```

Example

```
Student.find({ grade: { $gte: 8 } })  
  .then((students) => console.log(students))  
  .catch((err) => console.error(err));  
// Returns students with grade 8 or higher
```

find() with Projection

The second argument to `find()` is a projection, which controls which fields are included or excluded in the returned documents. Use 1 (or a field name string) to include a field and 0 to exclude it.

Syntax

```
Model.find(<filter>, { <field>: 1 })
```

Example

```
Student.find({}, { name: 1, grade: 1, _id: 0 })  
  .then((students) => console.log(students))  
  .catch((err) => console.error(err));  
// Returns only the name and grade fields for every student
```

find() with sort()

sort() orders the results of a find() query. Use 1 (or "asc") for ascending order and -1 (or "desc") for descending order on the given field. Because find() returns a chainable Query, sort() can be called directly on the result.

Syntax

```
Model.find(<filter>).sort({ <field>: 1 | -1 })
```

Example

```
Student.find()  
  .sort({ grade: -1 })  
  .then((students) => console.log(students))  
  .catch((err) => console.error(err));  
// Returns students ordered from highest to lowest grade
```

find() with limit() and skip()

limit() caps the number of documents returned by a query. skip() omits a given number of documents from the start of the results — commonly combined for pagination.

Syntax

```
Model.find(<filter>).skip(<n>).limit(<n>)
```

Example

```
Student.find()  
  .sort({ grade: 1 })  
  .skip(10)  
  .limit(5)  
  .then((page) => console.log(page))  
  .catch((err) => console.error(err));  
// Skips the first 10 results, then returns the next 5
```

findById()

A Mongoose-specific shortcut for finding a single document by its _id field. Equivalent to findOne({ _id: id }), but reads more clearly at call sites.

Syntax

```
Model.findById(<id>)
```

Example

```
Student.findById("64f1a2b3c4d5e6f7a8b9c0d1")  
  .then((student) => console.log(student))  
  .catch((err) => console.error(err));  
// Returns the document with that _id, or null if not found
```

findOne() Revisited

findOne() behaves like find() but always resolves to a single document (the first match) rather than an array. It still returns a chainable Query, so methods like sort() and select() can be applied before it resolves.

Syntax

```
Model.findOne(<filter>, <projection>)
```

Example

```
Student.findOne(
  { grade: { $gte: 8 } },
  { name: 1, _id: 0 }
)
  .then((topStudent) => console.log(topStudent))
  .catch((err) => console.error(err));
// Returns the name of the first matching student found
```